

SPHERE-PPL Forecasting Contest – NBN Atlas Species Occurrence Forecasting

Author: Alex Rabeau

Date: September 2026

Introduction

This report outlines a workflow for predicting species presence across the UK using environmental predictor data and a HistGradientBoosting classification model.

Occurrence records for the selected species are combined with gridded environmental predictors, while pseudo-absence locations are generated to provide negative examples for model training. The model is trained using historical observations (1970-2022) from UK regions outside a specified target region and subsequently validated on observations from the target region during more recent years (2023-2025).

The fitted model produces probabilities of species presence, with validation performance assessed using ROC AUC and Brier/MSE.

1. Load Data

We begin by loading processed occurrence datasets for nine species:

Euplagia quadripunctaria Fratercula arctica Haliaeetus albicilla Lutra lutra Pandion haliaetus Pipistrellus pipistrellus Sciurus vulgaris Triturus cristatus Vipera berus

```
euplagia_quadripunctaria = pd.read_csv(data_dir / "Euplagia_quadripunctaria/euplagia_quadripunctaria_processed.csv")
fratercula_arctica = pd.read_csv(data_dir / "Fratercula_arctica/fratercula_arctica_processed.csv")
haliaeetus_albicilla = pd.read_csv(data_dir / "Haliaeetus_albicilla/haliaeetus_albicilla_processed.csv")
lutra_lutra = pd.read_csv(data_dir / "Lutra_lutra/lutra_lutra_processed.csv")
pandion_haliaetus = pd.read_csv(data_dir / "Pandion_haliaetus/pandion_haliaetus_processed.csv")
pipistrellus_pipistrellus = pd.read_csv(data_dir / "Pipistrellus_pipistrellus/pipistrellus_pipistrellus_processed.csv")
sciurus_vulgaris = pd.read_csv(data_dir / "Sciurus_vulgaris/sciurus_vulgaris_processed.csv")
triturus_cristatus = pd.read_csv(data_dir / "Triturus_cristatus/triturus_cristatus_processed.csv")
vipera_berus = pd.read_csv(data_dir / "Vipera_berus/vipera_berus_processed.csv")
```

2. Prepare Occurrence Data

The selected species dataset is extracted from the collection of species-specific dataframes. The Event.Date field is converted using pandas datetime parsing.

```
occ = species_dfs[SPECIES].copy()
print("Occurrences:", occ.shape)

occ["Event.Date"] = pd.to_datetime(
    occ["Event.Date"],
    errors="coerce"
)
```

3. Construction of Presence and Pseudo-Absence Data

Known occurrence records are treated as presence observations and assigned a value of 1. The environmental predictors associated with these occurrence records are retained alongside the grid identifier, region and coordinates.

For each year, the corresponding gridded environmental dataset is then loaded from the environmental predictor directory. Grid cells containing known presences are excluded from pseudo-absence sampling.

Pseudo-absences are randomly sampled from the remaining grid cells. The number of pseudo-absences is controlled by an absence-to-presence ratio of three, meaning that three pseudo-absence observations are generated for every occupied grid cell. The random seed is fixed at 42 to make the sampling reproducible.

```
def get_presence_data(
    occurrences,
    year
):

    columns = ["grid_id", "Region", "Longitude..WGS84.", "Latitude..WGS84."] + predictors

    presence_data = occurrences[columns].copy()
    presence_data["presence"] = 1
    presence_data["year"] = year

    return presence_data

def generate_pseudo_absences(grid, presence_grid_ids, n_absences, rng):

    available = grid[
        ~grid["grid_id"].isin(
            presence_grid_ids
        )
    ].copy()

    sampled = available.sample(
        n=n_absences,
        random_state=rng
    ).copy()

    sampled["presence"] = 0

    return sampled
```

The resulting yearly dataset contains both presence and pseudo-absence observations, with a common set of environmental predictor columns.

4. Model Training

The training dataset covers 1970–2022. Observations from the target region are excluded from training.

For each training year, occurrence data outside the target region are used to construct the presence component of the dataset. Pseudo-absences are then generated from the corresponding annual environmental grid.

The model is implemented as a scikit-learn pipeline consisting of median imputation followed by a HistGradientBoostingClassifier.

```
# Training occurrences exclude the target region
yearly_occurrences = occ[
    (occ["Year"] == year)
    &
    (occ["Region"] != TARGET_REGION)
].copy()

# Create model
model = make_pipeline(

    SimpleImputer(
        strategy="median"
    ),

    HistGradientBoostingClassifier(
        max_iter=300,
        learning_rate=0.05,
        max_leaf_nodes=15,
        l2_regularization=1.0,
        random_state=RANDOM_SEED
    )
)
```

5. Model Validation

Validation is performed independently from model training using observations from the target region for the years 2023–2025.

For each validation year, occurrence records are restricted to the target region and combined with newly generated pseudo-absences. This ensures that the validation dataset represents both observed presences and sampled non-presence locations in the region that was excluded from training.

The fitted model generates a probability of presence for every validation observation:

```
val_probability = model.predict_proba(
    X_val
)[: , 1]
```

Two metrics are then calculated:

ROC AUC, measuring the ability of the model to discriminate between presence and pseudo-absence observations. Brier/MSE, calculated as the mean squared error between the observed binary presence values and predicted probabilities.

For *Euplagia quadripunctaria* in the East region, using training data from 1970–2022 and validation data from 2023–2025, the pipeline reports:

AUC: 0.879 Brier/MSE: 0.178